```sql
-- Question: Dream11 wants to identify users who are high-value players but not yet the top earners.
-- Find the second-highest total_winnings among all users and return the user_id, favorite_team, and total_winnings of users who earned that amount.
-- This will help the team target these users with offers to potentially convert them into top-tier customers.


Select user_id, favorite_team, total_winnings
FROM `user.user_activity`
WHERE total_winnings IN (SELECT MAX(total_winnings) FROM `user.user_activity`
                        WHERE  total_winnings < (SELECT MAX(total_winnings) FROM `user.user_activity`))



-- Select user_id, favorite_team, total_winnings
-- FROM `user.user_activity`
-- WHERE total_winnings IN (7000)

Select user_id, favorite_team, total_winnings
FROM `user.user_activity`
WHERE total_winnings  = 7000


-- Question: Dream11 wants to identify the total number of contests joined and contests won by Premium users only, so they can analyze the performance of their paid subscribers.

WITH temp_table as
(
 SELECT
 user_id,
 contests_joined,
 contests_won
 from `user.user_activity`
```

```
 WHERE subscription_status = 'Premium'
)

SELECT * from temp_table


-- Question: Dream11 wants to identify users with high referral counts (more than 5 referrals) who also have premium subscriptions, and then count how many such users have a win rate above the average win rate among all premium users, to assess the referral-driven engagement in the premium segment.


WITH premium_users as
(
 Select * from `user.user_activity` where subscription_status = "Premium"
)
,

high_referral_premium as
(
Select * from premium_users WHERE referral_count > 5
)

Select count(user_id)
from high_referral_premium
WHERE win_rate > (SELECT AVG(win_rate) from premium_users)


-- Show users who have contests_joined greater than average contests_joined

WITH base AS (
   SELECT contests_joined
   FROM `user.user_activity`
),
avg_val AS (
   SELECT AVG(contests_joined) AS avg_cj
```

```sql
   FROM base
)
SELECT user_id, contests_joined
FROM `user.user_activity`
WHERE contests_joined > (SELECT avg_cj FROM avg_val);


-- Question: Dream11 wants to identify highly engaged Premium users (session_duration
> 15) and high-referring non-Premium users (referral_count > 5) for targeted
campaigns.

-- Write a SQL query to return user_id, region, session_duration, referral_count, and
subscription_status for these groups, ensuring each user appears once, sorted by
user_id.

-- Objective: Identify two distinct user segments:
-- High-Engagement Premium Users: Users with session_duration > 15 and
subscription_status = 'Premium', indicating loyal users for premium feature
promotions.
-- High-Referring Non-Premium Users: Users with referral_count > 5 and
subscription_status != 'Premium', indicating influential users for upgrade incentives.
-- Returned: A list of user_id, region, session_duration, referral_count, and
subscription_status for users meeting either condition, with duplicates removed,
sorted by user_id.



WITH high_engagement_premium_users as
(
 SELECT
   user_id,
   region,
   session_duration,
   referral_count,
   subscription_status
 FROM `user.user_activity`
```

```
 WHERE session_duration > 15
 AND subscription_status = 'Premium'
),

high_referring_non_premium_users as
(
 SELECT
   user_id,
   region,
   session_duration,
   referral_count,
   subscription_status
 FROM `user.user_activity`
 WHERE referral_count > 5
 AND subscription_status != 'Premium'
)

SELECT * FROM high_engagement_premium_users
UNION DISTINCT
SELECT * FROM high_referring_non_premium_users



-- Question: Dream11 wants to compare engagement for users from Delhi or with
favorite_team 'Delhi Capitals'.

-- Write a query to return the group label (region or favorite_team), total users,
total winnings, and average session_duration for these segments, allowing duplicates.

-- Objective: Analyze engagement for two segments:
-- Users from the Delhi region.
-- Users whose favorite_team is Delhi Capitals.
-- Returned: A dataset with group_label ("Region - Delhi" or "Fans - Delhi Capitals"),
total_users (count of users), total_winnings (sum of winnings), and
avg_session_duration (average session duration) for each segment.
```

```sql
WITH delhi_region_users AS
(
 SELECT
 user_id, total_winnings,session_duration, "Region-Delhi" as group_label
 FROM `user.user_activity`
 WHERE region = "Delhi"
 AND total_winnings is not null
 AND session_duration is not null
),

delhi_capitals_fans AS (
   SELECT
       user_id,
       total_winnings,
       session_duration,
       'Fans - Delhi Capitals' AS group_label,
   FROM `user.user_activity`
   WHERE favorite_team = 'Delhi Capitals'
   AND total_winnings IS NOT NULL
   AND session_duration IS NOT NULL
),

all_data as
(
Select * from delhi_region_users
UNION ALL
Select * from delhi_capitals_fans
)


Select
group_label,
COUNT(*) as total_users,
SUM(total_winnings) as total_winnings,
AVG(session_duration) as avg_session_duration
from all_data
```

```
GROUP BY group_label



-- UNION DISTINCT VS UNION ALL

WITH table_1 as
(Select favorite_team from `user.user_activity`
LIMIT 2
),

table_2 as
(Select favorite_team from `user.user_activity`
LIMIT 2
)

Select * from table_1
UNION ALL
Select * from table_2
```